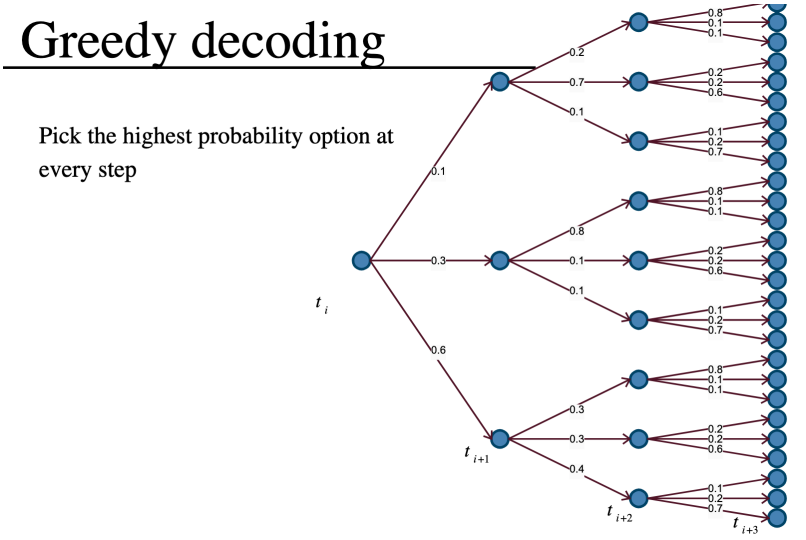
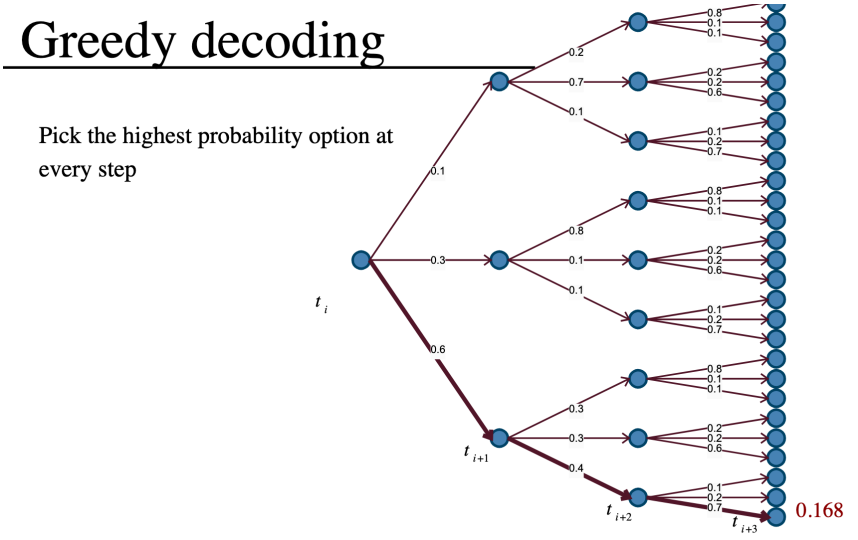
LECTURE 11

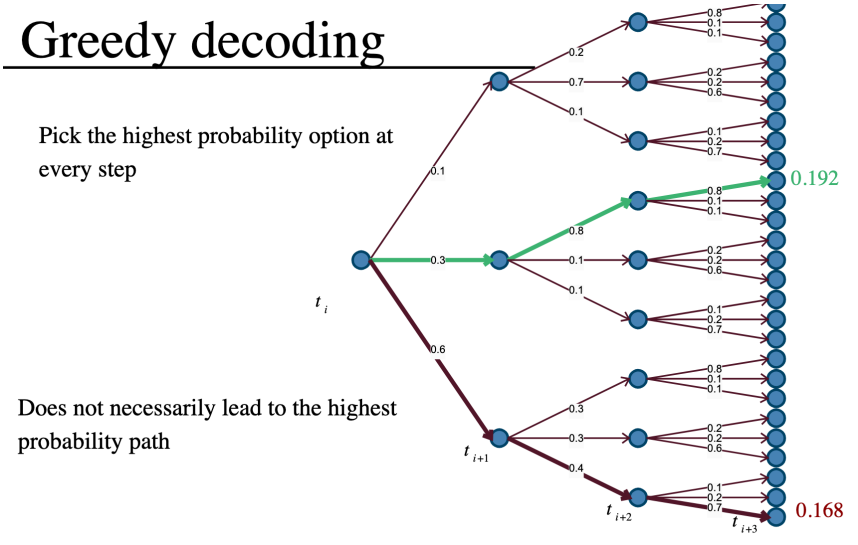
Sampling from Large Language Models

In the last lecture, we saw how to construct (large) language models, and some of the basic ideas behind their training. In this lecture, we will see how the models can be used for program synthesis.

11.1 Basic sampling



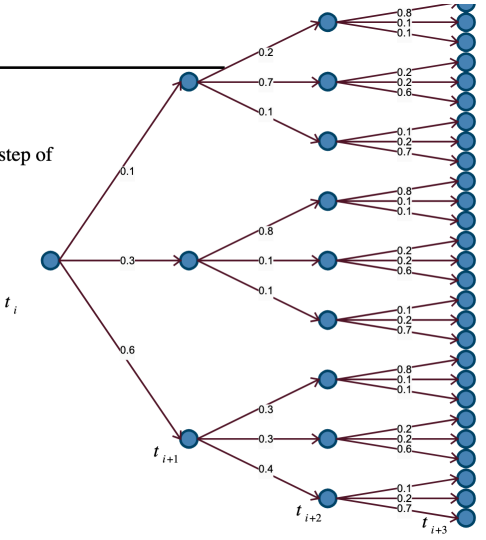

Greedy decoding
Pick the highest probability option at every step
0.168


Greedy decoding
Pick the highest probability option at every step
Does not necessarily lead to the highest probability path
0.192
0.168

Beam search

Define a beam size K

Keep K paths down the tree at each step of search

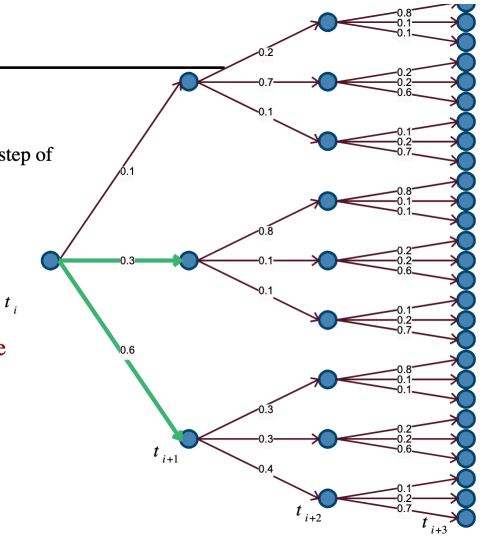


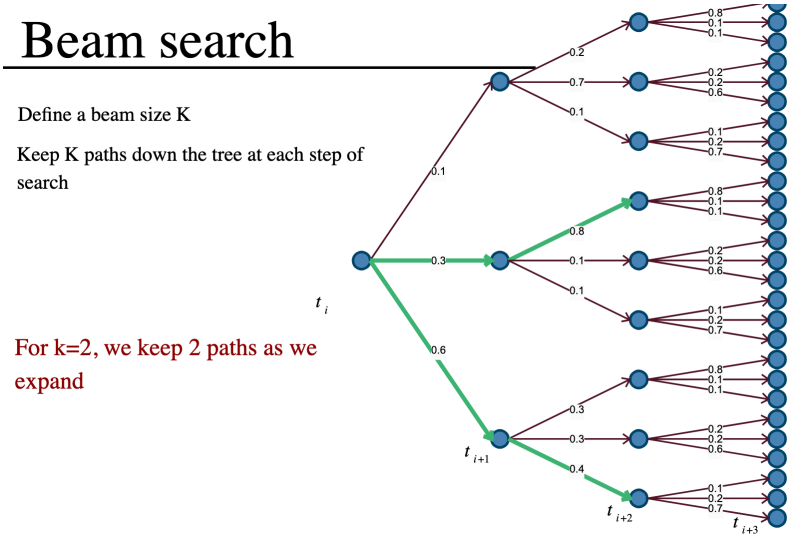
Beam search

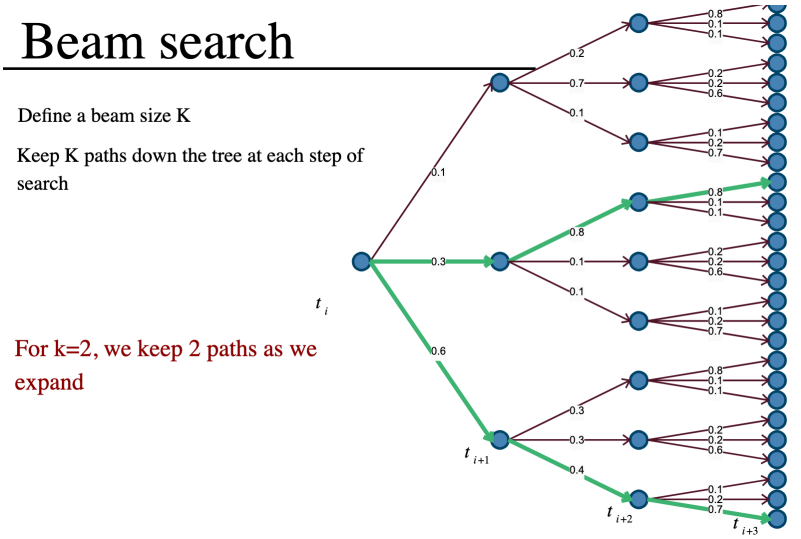
Define a beam size K

Keep K paths down the tree at each step of search

For k=2, we keep 2 paths as we expand







Fundamentally, language models parametrize distributions $p_\theta(\bar{t})$ over sequences of tokens $\bar{t} = t_1, \ldots, t_n$, with a key property that because of the way the models are trained, they learn not just the joint distribution over sequences of tokens, but also the conditional distribution over the next token given the previous ones. It is important to note that in order to obtain probabilities, the output of the model is passed through a softmax layer, which converts the raw scores produced by the model into a probability distribution over the tokens in the vocabulary. The softmax includes a *temperature* parameter τ, which is used to control the entropy of the distribution.

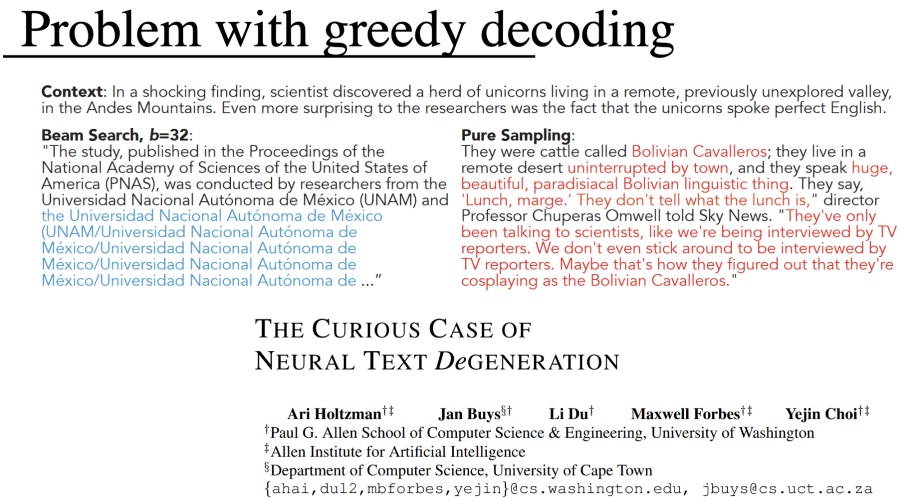
$$softmax(z_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

When $\tau \to 0$, this distribution approaches the delta distribution on the most likely token, and when $\tau \to \infty$, it approaches the uniform distribution over all tokens.

For most synthesis problems, we start with a prefix of tokens $t_1, \ldots, t_k$ which contains all the evidence that we want to condition on. This is the prompt, and may include information such as a natural language description of the task, some input-output examples, and perhaps some pseudocode or other information relevant to describe the task.

One naive way to use the model is to start with the prompt and then pick the most likely next token $t_{k+1} = \arg\max_t p_\theta(t|t_1, \ldots, t_k)$, then append this token to the prompt and repeat until we reach the end of the sequence. This is known as *greedy decoding*, and while it is simple, it has been found to produce bad results in practice; it has been observed in particular to produce stuttering and other artifacts not present in the training data [1].

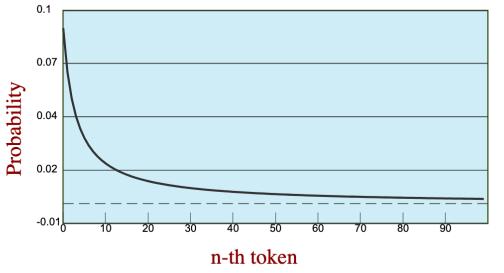
Some of the problems with greedy decoding come from the fact that it is greedy, and therefore may not actually be producing the highest probability sequence. One way to address this is to perform a *beam search*. The key idea behind beam search is to maintain a *beam* of the k most likely prefixes at each step of the sampling process. At each step, we expand each prefix in the beam by one token, and then keep only the k most likely prefixes among all the expanded prefixes. This way, we can explore multiple possible continuations of the prompt, while still keeping the search space manageable. Unfortunately, beam search leads to the same types of problems as greedy decoding.



THE CURIOUS CASE OF
NEURAL TEXT *De*GENERATION

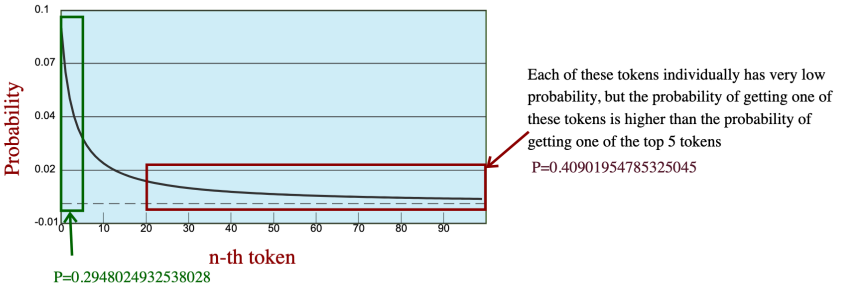
Ari Holtzman[†‡] **Jan Buys**[§†] **Li Du**[†] **Maxwell Forbes**[†‡] **Yejin Choi**[†‡]
[†]Paul G. Allen School of Computer Science & Engineering, University of Washington
[‡]Allen Institute for Artificial Intelligence
[§]Department of Computer Science, University of Cape Town
{ahai,dul2,mbforbes,yejin}@cs.washington.edu, jbuys@cs.uct.ac.za

What is the problem with sampling?



With many tokens, the probability of a low probability token is higher than the probability of a high probability token.

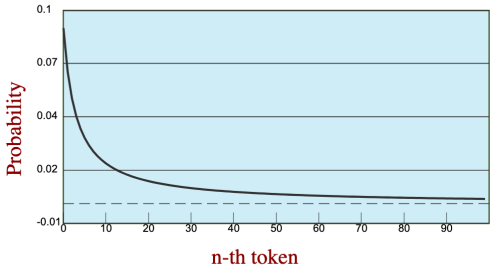
What is the problem with sampling?



With many tokens, the probability of a low probability token is higher than the probability of a high probability token.

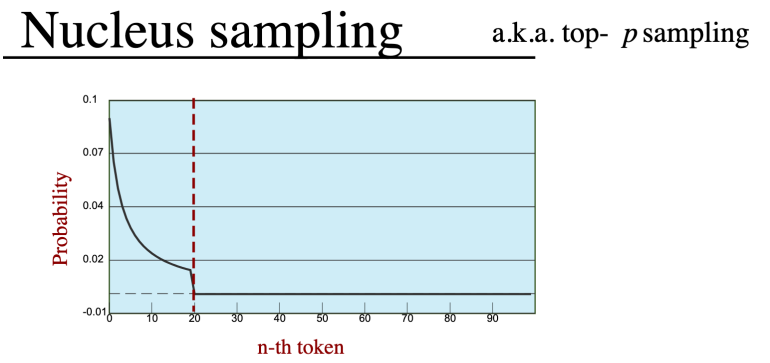
Nucleus sampling

a.k.a. top- *p* sampling



Idea: eliminate the long tail of low-probability values.

Sample from the modified distribution.



As it turns out, despite the fact that the output of the model is meant to represent the probability distribution over the next token, making the best use of this information to produce a high-quality output is not as straightforward. For example, another approach is to simply sample directly from the distribution $t_{k+1} \sim p_\theta(t|t_1, \ldots, t_k)$. As was also documented by Holtzman et al., though, this approach also produces poor results. The problem is that the distribution contains a long tail of low-probability tokens, so even though these tokens individually have very low probability, the probability of sampling one of them is quite high, leading to incoherent outputs. To address this, Holtzman et al. proposed a sampling strategy called *nucleus sampling*, where instead of sampling from the full distribution, we first restrict the distribution to the smallest set of tokens whose cumulative probability is at least p (for some parameter p), and then sample from this restricted distribution.

11.2 Search

Everything we have discussed so far comes from the world of natural language processing, where the goal is to produce text that is coherent, but it is difficult to mechanically evaluate the quality of the output. In contrast, program synthesis can frequently benefit from hard constraints on the outputs, such as passing a set of unit tests. Additionally, programming languages tend to have much stronger syntactic constraints than natural languages, which can be exploited to identify invalid outputs. As a result, synthesizing a program that satisfies the hard constraints often requires generating many candidate programs and then filtering them based on the constraints.

Notation. We will use the notation $p_\theta(\cdot)$ to denote the distribution over strings induced by a language model with parameters θ. For hard constraints

C, we will use the notation $p_\theta(\cdot|C)$ to denote the distribution over strings that satisfy the constraint C, and we will use $1(s \models C)$ to denote the indicator function that returns 1 if the string s satisfies the constraint C and 0 otherwise. With some abuse of notation, when we write $p_\theta(\cdot; p)$ we will mean the distribution over continuations of the prompt p. We will also use the notation ++ to denote concatenation of prompts.

11.3 Rejection sampling

The simplest way to obtain a sample which adheres to some hard constraint C is to use *rejection sampling*. Algorithmically, we can sample from the distribution $p_\theta(\rho|C)$ as follows:

1. Sample $\rho \sim p_\theta(\cdot)$.
2. If $1(\rho \models C) == 1$ return ρ; else restart from step 1.

This algorithm is very simple, and it works well when the constraint C is easy to satisfy (ie., when it is likely that we will find a satisfying program ρ in the first few tries). However, it can be very inefficient when the constraint is hard to satisfy, as we may have to sample many times before we find a program that satisfies the constraint. This issue is particularly apparent when we are sampling from a flexible distribution such as that induced by a large language model, where the space of possible programs is very large.

To address this, we can try to bias the sampling process towards the programs that are more likely to satisfy the constraint; indeed, one of the strengths of the language modeling paradigm is that it is very simple to turn the constraint into some textual prompt that can be used to condition the model. For example, we could *approximate* the distribution $p_\theta(\rho|C)$ by encoding C as a prompt p_C and sampling from the distribution $p_\theta(\rho; p_C)$. How good of an approximation this is fundamentally depends on our model's capacity; however, even the best models will be imperfect, and thus we will still need to use some clever sampling strategy.

11.4 Filtering based on observational equivalence

An interesting early example of a system which went one step beyond rejection sampling is AlphaCode [2]. AlphaCode sought to solve problems from programming competitions; at the time, generalist models were not yet very competent programmers, so instead AlphaCode begun by taking a language model which had first been pre-trained on a large set of programs from Github, and then finetuned the model on a dataset of programming competition problems. Despite this specialization, performance was still not very good, so AlphaCode

introduced a novel sampling strategy to improve the quality of the solutions. First, a very large number of candidate programs (in its most aggressive mode, 1 million samples per task) were sampled from the model. Then, AlphaCode performed rejection sampling on a set of *public* unit tests, which were provided as part of the problem description. This turned out to not be enough, however, as many of the sampled programs were still incorrect; when evaluated on the hidden test cases, which were not provided as part of the problem description, many of the programs would fail. As a result, AlphaCode introduced a second filtering step, where it would take the set of programs that passed the public tests and then cluster them based on observational equivalance on synthetic input data. Finally, only one program from each cluster was returned. Thus, the final output of AlphaCode was a small set of unique programs that passed all the public tests, and which were also distinct from each other in terms of their behavior on the synthetic input data, increasing the chances that at least one of them would pass the hidden tests.

11.5 Self-repair

As language models became more powerful, it became possible to use them to not just generate continuations of programs, but also to *repair* programs that were known to be incorrect. This begged the question of whether it was possible to increase the quality of the sampled programs by using the model to iteratively improve them. A simple implementation of this idea is as follows:

1. Sample a program $\rho \sim p_\theta(\cdot; p_C)$.
2. If $1(\rho \models C) == 1$ return ρ; else, let $e = C(\rho)$ be the error of ρ with respect to the constraint C.
3. Sample a repaired program $\rho \sim p_\theta(\cdot; p_C + +\rho + +e)$.
4. Repeat from Step 2.

This procedure is known as *self-repair*, *self-debugging* or *self-refinement*. The idea here is that if the model is able to leverage the information contained in the error e to repair ρ, then we will be able to obtain a program that satisfies the constraint C with fewer samples. However, work by Olausson et al. showed that the efficacy of this approach depends highly on the model's ability to link the error e to the constraint C, and as of late 2023 only the largest models were able to do this reliably. Furthermore, while we have presented a simplistic version of the algorithm that only considers one initial sample $\rho \sim p_\theta(\cdot; p_C)$, in practice it was found that it was often more useful to sample multiple candidates $\rho_1, \ldots, \rho_n \sim p_\theta(\cdot; p_C)$ before attempting repair.

The self-repair procedure can be seen as a form of *iterative refinement*, where we iteratively sample a solution and then refine it based on information we receive from the environment. Nowadays, self-repair has been absorbed by the more general iterative framework employed by so-called *LLM agents*, where interactions with the environment (and iterating on the solution) play a central role. We will talk more about agents in Lecture 16.

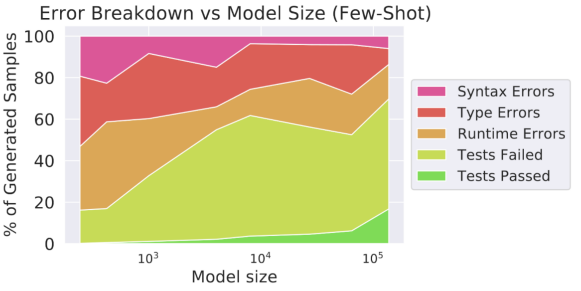
11.6 Constrained Decoding

Sometimes, we may not have any hard constraints on the semantics of the program we want to sample, but we may still want to ensure that the sampled program satisfies some other type of hard constraint. In particular, if we know which programming language we would like the solution to be in, we may as a first step at least want to ensure that the sampled program is syntactically valid in that language. In this special case, we can use a technique called *constrained decoding* to ensure that the sampled program is syntactically valid.

The high-level idea is simple. Let $\mathcal{L}$ be the set of all strings which are valid programs in the target programming language. Note that $\mathcal{L}$ may be infinite, but that is no concern; since we have a grammar for the programming language, it is easy to check whether a given string s belongs to $\mathcal{L}$ by parsing it. Thus, we already have a procedure for checking whether a string is a valid program, and we can use this to filter out invalid programs with rejection sampling. There are two concerns with this approach, however. The first concern is: is this even worth it? In an early paper by a team of Google researchers [3], they showed that as models got larger, syntax errors became quite rare. Moreover, even when there is a syntax error, it is not clear that avoiding the syntax error will lead to a correct program. When the model makes a syntax error, it is likely because the task is so out of distribution that the model has little idea of how to solve the problem, so correcting the syntax simply replaces a garbage solution with syntactically valid garbage. It is important to note, however, that this reasoning only applies to programs in high-resource languages (the results in the Google paper were for Python); for low-resource languages, syntax errors can still be more common, and avoiding them can help the model better leverage knowledge it is able to transfer from high-resource languages.

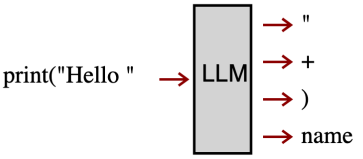
Error breakdown

As the model size grows, syntax and type errors become less and less relevant

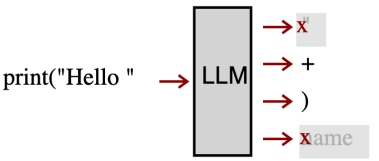


From Austin et al, "Program Synthesis with Large Language Models", 2021

Constrained decoding: Synchromesh

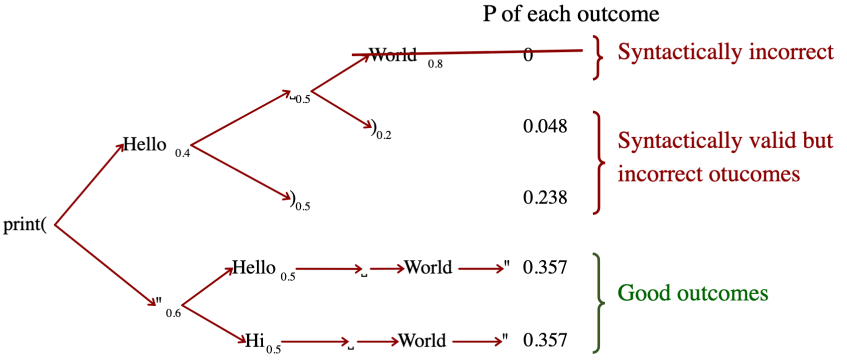


Constrained decoding: Synchromesh



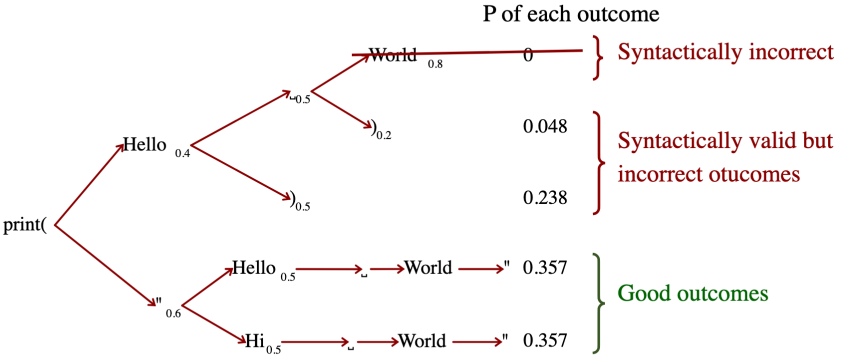
Avoid gramatically incorrect outputs by filtering out from the output distribution any token that does not lead to a valid program prefix.

Simple constrained decoding

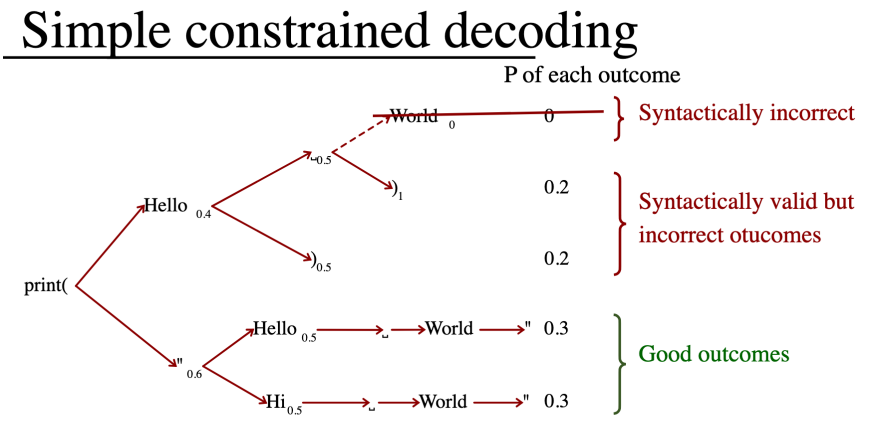


If we remove the syntactically incorrect path, the probabilities of all other paths are adjusted.

Simple constrained decoding



The probability of a good outcome rises from 0.6 to 0.714



But if we avoid the syntax error by locally avoiding the choice that directly leads to the error, the probability of a good outcome stays the same.

The second concern is efficiency. We can of course use rejection sampling to filter out invalid programs, but can we do better?

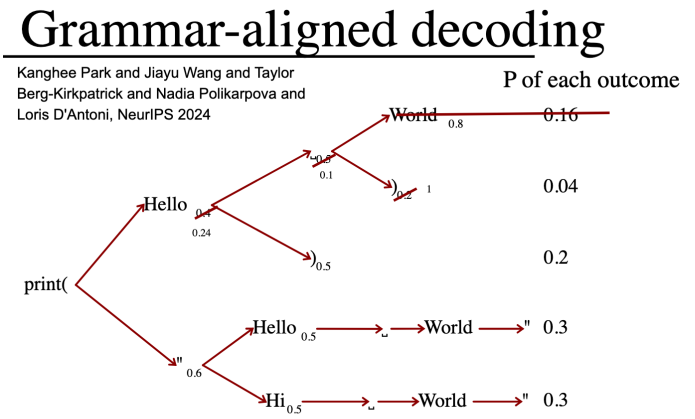
The answer to the first question is yes. Several papers in 2021 and 2022 showed that constrained decoding can improve the quality of the generated programs, especially for low-resource languages. One of the earliest examples of using constrained decoding for program synthesis is the work of Scholak et al. [4], which focuses on generating SQL queries from natural language.

Another notable example is the Synchromesh project [5]. By using the *prefix closure* $\mathcal{L}^c = \{u \mid \exists v.uv \in \mathcal{L}\}$ of $\mathcal{L}$, i.e., the set of all strings that can be extended to a valid program, we can ensure at each step of the sampling process that the sampled string is a valid prefix of a program in $\mathcal{L}$. It is not immediately obvious, but it turns out that answering the question of whether a string s is in $\mathcal{L}^c$ can be done somewhat efficiently; Poesia et al. found that their implementation could ensure syntactically valid programs with only an 8% overhead compared to sampling from the model without any constraints.

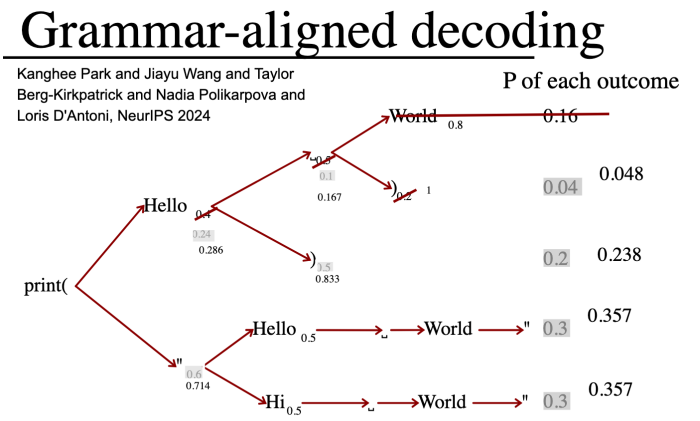
Since Syncomesh, the idea of constrained decoding has been adopted by a number of other systems. The idea has been generalized beyond syntax to richer semantic constraints. For example, Mundler et al. in their work on type-constrained code generation have shown how to incorporate type information into the sampling process [6]. They focus on TypeScript, and they show that for synthesis and translation problems, they can reduce compilation errors in the generated code by half, although correctness is only improved by between 3.5 and 5.5%. The picture is better for repair problems, where their approach "enhances functionally correct repair of non-compiling code relatively by 37% on average."

11.7 Unbiased constrained decoding

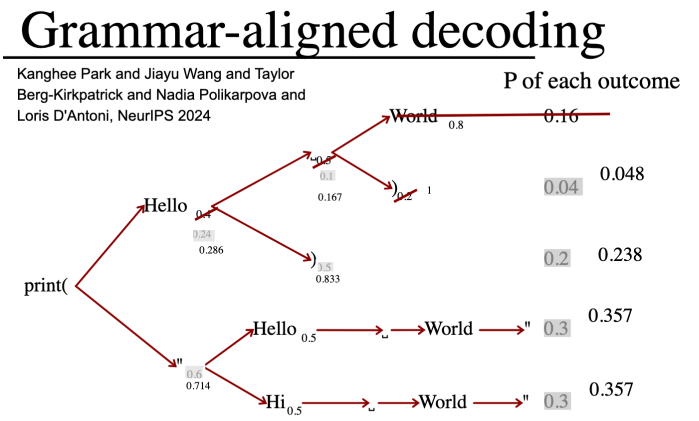
A general problem with constrained decoding approaches that locally steer the model away from invalid tokens is that they do not actually sample from the distribution $p_\theta(\cdot|C)$. The issue is illustrated by the figure above. The problem is that the model may make a bad decision early on, but the bad decision may not lead to a syntax error until much later. By that point, avoiding the syntax error may simply be shifting some of the probability mass from one invalid program to another similar program that is syntactically valid but incorrect. More generally, we see that simply avoiding the syntax errors is not the same as sampling from the distribution $p_\theta(\cdot|C)$.



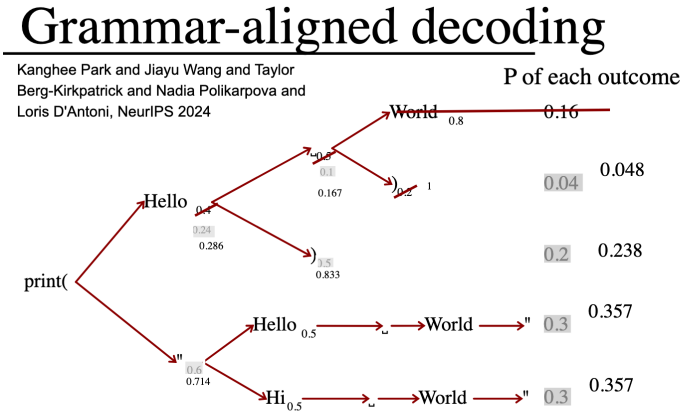
Key idea: all nodes upstream from a node with its choices reduced must have its probability adjusted to account for the lost probability mass.



After renormalizing the weights, we get the correct probabilities for each outcome.



Challenge: how do you know how much to adjust before traversing the full tree?



You don't, the idea is to build these adjustments incrementally with every sample taken from the model. This only works in applications where you are sampling repeatedly from the model.

This issue was first documented by Park et al in their work on Grammar-aligned decoding [7]. The key observation behind this work is that it's not enough to simply avoid invalid tokens; in order to get a correct distribution, the probability of valid tokens must also be adjusted. The problem is that these adjustments require foresight about how much probability mass will be lost from the search tree in the future, which is not available when making local decisions.

The solution proposed by Park et al is to build up these adjustments incrementally as one samples repeatedly from the model. This only works if one is trying to sample repeatedly from the model, which is a reasonable assumption in the context of program synthesis. The algorithm is probably not the last word on this problem, as it involves non-trivial overheads and is slow to

converge to the true conditional distribution, but it is likely that we will see the insights from this work being built into future constrained decoding algorithms.

References

[1] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1904.09751.

[2] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL https://www.science.org/doi/abs/10.1126/science.abq1158.

[3] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021. URL https://arxiv.org/abs/2108.07732.

[4] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.779. URL https://aclanthology.org/2021.emnlp-main.779/.

[5] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=KmtVD97J43e.

[6] Niels Mündler, Jingxuan He, Hao Wang, Koushik Sen, Dawn Song, and Martin Vechev. Type-constrained code generation with language models. In *46th ACM SIGPLAN International Conference on Programming Lan-*

guage Design and Implementation (PLDI '25), pages 601–626. ACM, 2025. doi: 10.1145/3729274. URL https://doi.org/10.1145/3729274.

[7] Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. Grammar-aligned decoding. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/2bdc2267c3d7d01523e2e17ac0a754f3-Abstract-Conference.html.